

Category 1: Mobile Automation

Question:

1- Explain how you would handle testing for different device screen sizes and orientations (portrait vs landscape) when automating mobile tests. What strategies or tools would you use to ensure your tests are robust across various devices?

- Emulators and simulators like Android Studio and Xcode play a crucial role in our testing strategy, allowing us to simulate a wide range of devices.
- We also leverage real devices such as Samsung Fold 3, Flip 4, A53, Pixel 7, iPhone 13 Pro, and 15 Pro Max to ensure accurate testing on actual hardware. Responsive design testing frameworks help us write tests that adapt to different screen sizes.

Category 2: Web Automation

Question:

1-What are the best practices for handling dynamic web elements (e.g., elements whose IDs or classes change frequently) in test automation? How can you ensure your automated tests remain stable despite these changes?

- Instead of relying on IDs or classes, look for elements that stay the same, like their position on the page (using relative XPaths) or unique attributes (like data-testid).
- Use "waits" to give the page time to load before trying to interact with elements.
- Use the Page Object Model to keep your element locators in one place, making it easier to update them when things change.
- Create locators that can adapt to small changes, like using regular expressions or parameterized locators.

2-You have a login Page, and when clicking on the login button after inserting the information nothing happened. You reported the defect and the Frontend and Backend team told you that there is no issue from their side. What are the steps needed to identify the root cause and communicate the issue? and list all possible scenarios

- **Frontend:** Thoroughly inspect the frontend code for JavaScript errors, incorrect event listeners, or missing form submissions.
- **Backend:** Test the backend API directly using Postman to confirm it's functioning correctly.
- **Investigate in Splunk:**
 1. Utilize correlation IDs to trace the request through the system.
 2. Analyze logs from each API and the gateway to pinpoint the exact point of failure.
 3. Examine log entries for error messages, latency spikes, or unexpected behavior.
- Use browser developer tools to monitor network requests.
- Verify request/response details, including status codes, headers, and payloads.

Collaboration:

- Schedule a meeting with frontend and backend developers to present findings, including Postman results and Splunk analysis.
- Jointly debug the issue using browser developer tools and debugging breakpoints.

Possible Scenarios:

1. The API gateway may be blocking requests, experiencing latency, or misrouting traffic.
2. Frontend sending requests to the wrong endpoint.
3. Interfering with communication between frontend and backend.
4. Issues with session handling or authentication cookies.

Category 3: API Automation

Question:

Explain how you would test the **rate limiting** feature of an API. How would you design an automated test to verify that the API responds with the correct status code when the rate limit is exceeded?

To test API rate limiting, we simulate a large number of requests in a short period of time to see how the API handles the load. Initially, we expect the API to respond successfully to the requests. However, once the rate limit is exceeded, the API should return a specific error code, such as 429, indicating that the rate limit has been hit. To automate this process, we can use libraries like OkHttp in Java to send a burst of requests to the API and check for the expected error responses. This kind of testing helps ensure that the API can maintain a stable service by preventing abuse and making sure that all users have fair access.